

Predicting when training-free sparse prefill pays on real host-GPU pairings: a component-composed critical-path model and a test of attention recall as a quality proxy. Research gaps and methodology (v7)

4 Oct 2026 · @mh · v7: v6 revised in response to a fourth referee report (verdict: promising but needs methodological rework); separates link cost in the recursion, replaces frequency bands with a measured host-speed probe, adds recall by layer depth, a launch-queue sensitivity analysis and a host-memory contention check, and makes G2 and G3 the lead contributions

Status and scope

This is a research direction and a pre-registration draft. The new study has taken no measurements yet.

- v4 keeps v3's question and gaps and replaces its validation design. v3 validated the cost model partly with synthetic host delay injected on one machine. The referee judged that fatal, because a busy-wait does not reproduce a different CPU's caches, memory system, PCIe link or clock behaviour. v4 validates only on physically distinct host-GPU pairings, and removes delay injection from every hypothesis.
- v7 answers a fourth referee report. Link cost becomes its own term in the recursion; cloud CPU frequency, which the guest cannot fix or reliably read, is controlled through a timed host-speed probe with acceptance windows; recall is also reported by layer depth; the launch-queue term gets a sensitivity analysis; host-memory contention with copies is measured; and G1 is presented as the validation question, with the novelty resting on G2 and G3. Two of the referee's claims are rejected with reasons below. After the report, four further papers were read in full (LLMEdger, Less Is More, The Sparse Frontier, and APEX again); SparkV was withdrawn by its authors and is no longer cited. Five more papers were then read in full (the APEX simulator, CompactAttention, oracle-guided sparse prefill, AB-Sparse and SKIP/TKLQT); the changes they caused are listed in their own section below.
- v6 answered a third referee report. G1 is narrowed and reframed against learned cross-device latency predictors; a learned component-feature predictor becomes a baseline the model must beat; the execution model gains a finite launch-queue term and a measured copy-contention check; NUMA placement, pinned versus pageable memory and unified-memory faults are controlled; and an oracle-mask ablation separates estimator cost from synchronisation cost.
- v5 added four things to v4: newly verified literature (SparkV, since withdrawn by its authors; CompactAttention, AB-Sparse, oracle-guided sparse prefill, Token Sparse Attention); secondary predictions about how the crossover moves with GPU speed relative to the host; a scoped chunked-prefill extension; and three AWS pairings on AMD hosts (L40S, single H100, optionally A10G), which the researcher's budget now covers. The second reviewer's direction as a whole was rejected; the reasons are recorded below.
- The pre-registration is drafted but unlocked. It binds only after the scoring code, its tests, two independent reviews and a dated, tagged lock commit.
- Banked evidence is unchanged: the earlier oracle-mask study on three GPUs, and the T4 pilots on an L4 (1–3 October 2026).
- Anything marked "not built" or "not run" is a plan. Tolerances shown as *[set before lock]* are deliberately open.
- The target is an IEEE Transactions-level journal. Nothing here claims it reaches that bar.

Response to the v3 referee report

Referee point	Response	Change in v4
Fatal: synthetic host-delay injection used to validate prediction on unmeasured pairings.	Accepted	Removed from every hypothesis. Validation uses physically distinct pairings only (four in v4, seven in v5), listed under "Hardware pairings". Delay injection survives only as an optional check of the timing instrument and is never cited as evidence for prediction. v3 already planned held-out real pairings, but with two or three hosts it leaned on injection to fill the gap; that dependency is gone.
Confounders: PCIe and memory bandwidth, cache sizes, CPU boost clocks.	Accepted	The model now has three component classes: host-platform costs (measured on the target host, so its caches, memory and clocks are inside the measurement), device costs (measured on the target GPU), and link costs (measured per pairing by microbenchmarks: transfer bandwidth, 1-byte copy, sync round trip, PCIe generation and width). Separability of the first two is a hypothesis (H0), tested on real pairings. Clock and topology controls are listed under "Confounders and controls".
Major: device time from graph replay for capturable arms but traces for the rest.	Accepted	One method for all arms: each operation replayed in isolation on the GPU under Nsight Systems (TaxBreak's isolation-replay protocol), with the arm's recorded masks. This never runs the composed workload, so the prediction stays non-trivial. Graph replay of the whole prefill is a cross-check only, on capturable arms. Ground-truth end-to-end time always comes from untraced runs, and profiler overhead is reported.
Minor: recall sufficiency (H4) tested only at block size 128.	Accepted, bounded	H4 is tested at block sizes 32, 64 and 128 on the primary cell through a quality-only reference path (masked attention, untimed). The fast kernel stays at 128; block 16 is added only if the reference path supports it.
G4 (falsifiable crossover) is a methodological commitment, not a gap.	Accepted	G4 removed from the gaps. The crossover forecast survives as hypothesis H3, a consequence of the cost model, and pre-registration of predictions is listed as control K9.
Critical-density relation dilutes the focus.	Accepted	Reported descriptively; no hypothesis.
Hardware scope too narrow for a claim about "host-GPU pairings".	Accepted in part	The claim is restricted to the measured host platforms and GPUs, and each pairing is reported individually. With few pairings no pooled significance is claimed. The model and family scope is kept and stated as a limit.
APEX (CPU-GPU overlap prediction) weakens G1/G2.	Accepted	Added to positioning. APEX predicts CPU and GPU subtask times to schedule decode offloading; it does not predict end-to-end prefill on unmeasured pairings or model data-dependent synchronisation inside a step. Now read in full: IEEE IPDPS 2026, DOI 10.1109/IPDPS65963.2026.00099. Its "analytical model" is a scheduling inequality built from offline profiles of the machine it runs on, evaluated on two machines (Xeon Gold 6342 with A10, Xeon Gold 6130 with T4). Its deferred synchronisation shows that where a CPU-GPU synchronisation sits changes exposed time, which supports the premise of G2; it does not predict time on hardware it has not profiled.
DuoAttention is a useful baseline.	Accepted, conditional	Added as a head-differentiated arm (retrieval heads dense, streaming heads sink plus local) through the existing per-head selector, only where the authors' released head patterns exist for a study model. Its head identification uses an optimisation on synthetic data with frozen weights, so it is labelled "calibrated", not training-free.

Compute HDBI alongside the model.	Kept and strengthened	HDBI is a diagnostic, not a predictor, so the model is compared against two predictive baselines built from the same measured components: an overlap-free sum and KernelSight-LM's step form (an overlap floor plus a serial tail, with every synchronisation counted in the tail).
-----------------------------------	-----------------------	---

Response to the second reviewer's direction

A second reviewer proposed predicting sparse-prefill profitability from measured hardware parameters on a panel of at least six pairs across four GPU generations. It asks the same question as v4, so it was reviewed as an alternative to v4 rather than as an addition, and rejected as a replacement. The parts that survived are folded in.

Element of the direction	Decision	Reason and change in v5
Exposed host time as a per-layer maximum, $E(S) = \sum \max(0, t_{\text{host}} - t_{\text{gpu}})$	Rejected as the model	This is the per-step overlap floor already used by KernelSight-LM and already the baseline v4's recursion must beat. It cannot express where synchronisations fall inside a step (G2).
Parameters measured on every pair; one or two kernel-efficiency constants fitted and carried across generations	Rejected	Leave-one-pair-out is then nearly trivial, and leave-one-generation-out extrapolates sparse-kernel efficiency across architectures, which SparKV reported analytical models cannot capture. v5 keeps measured device components and tests composition and transfer explicitly. [v7 note: SparKV has since been withdrawn by its authors and is no longer cited; the same point now rests on KernelSight-LM's finding that attention-kernel efficiency does not transfer across GPU generations.]
Predictions indexed by GPU generation	Adopted, restated	Generation is the wrong axis: the L4 (Ada) is slower than the A100 (Ampere), and the earlier study found the L4 hides host work because it is slower. The predictions are restated by GPU speed relative to the host and added as H5.
Serving effects (per-chunk re-selection)	Adopted, scoped	CompactAttention reports XAttention and SeerAttention often slower than dense under chunked prefill. Added as an extension on one pairing; the recursion handles repeated per-chunk estimator calls without a new model.
Larger hardware panel; one GPU on two hosts	Adopted, verified	Seven pairings instead of four, all with host CPUs checked against provider announcements: three GPUs on one AMD host platform and the H100 on two hosts.
Quality measured once on one GPU with the dense control replicated on a second	Adopted	Accuracy does not depend on the GPU beyond numerical noise, which the exact dense-reproduction check bounds.
Quality only as a constraint (no recall-sufficiency test)	Rejected	Removes G3, leaving one contribution.
Batch above 1 out of scope because "the ratio does not change with batch"	Rejected	Per-launch host cost is fixed while device work grows with batch, so exposed host time shrinks; the direction's own model implies the ratio changes. v5 keeps batch 1 and 4.
"Considered and rejected" table; reviewer questions on a design contribution and on negative results	Adopted	Added below.
Unverified citations (Task-KV as IEEE TPAMI 2026; FlashPrefill V2, SparDA, EntropyInfer, PSA, cross-layer routing; Token Sparse Attention's $0.97\times$ vs $0.66\times$)	Held	Not used until read. Token Sparse Attention itself is confirmed (ICML 2026).

Response to the third referee report

Referee point	Response	Change in v6
G1 is not novel: component-based prediction of end-to-end latency on unseen CPU-GPU pairs exists (CTFusion: Gennai, Ohsita and Shimonishi, IEEE FLTA 2025).	Accepted	Confirmed and read in full (DOI 10.1109/FLTA67013.2025.11336244). CTFusion predicts end-to-end latency, including host preprocessing, host-device transfer, device time and their interaction, on unseen CPU-GPU pairs. It uses host, transfer and device micro-benchmarks plus bottleneck ratios, fed to LightGBM, on 25 configurations in which every GPU is paired with at least two CPUs. It reports 17.6% mean absolute percentage error on unseen hardware: 14.1% on RTX 4070 systems and 30.1% on the one A100 server. G1 as first framed is therefore not novel. What CTFusion leaves open: its workload is object-detection inference whose host term is input preprocessing throughput, not per-operation dispatch, data-dependent mask building and synchronisation inside a forward pass; its host-device interaction is learned implicitly rather than modelled; it needs 20 training systems; and it gives no uncertainty intervals. G1 is reframed as a comparative question, CTFusion-style features become the learned baseline, and the related learned-predictor line (nn-Meter, HELP, CloserToMe) is cited as well. An earlier draft of this response wrongly said the paper could not be found; it was found once the researcher supplied the PDF.
G2 overlaps with LLMervingSim 2.0, which models synchronisation and data dependencies in execution graphs.	Accepted	Read in full: IEEE ISPASS 2026, DOI 10.1109/ISPASS69572.2026.00012. Its execution DAGs encode data dependencies, synchronisation and communication between devices and memory tiers, built on ASTRA-sim and Chakra. Operator latencies come from a one-time profile per model and device; validation is on three platforms (RTX A6000 server, 8-GPU H100 system, TPU v6e), on vLLM serving traces. It does not model host dispatch or GPU-to-CPU synchronisation driven by the data, and it does not predict a model-device pair it has not profiled. The referee is right that dependency-and-synchronisation modelling exists. G2 is narrowed to a per-operation model of measured components that represents data-dependent mask generation and its synchronisations inside a prefill step, validated on real pairings with predictions committed first.
Major: the recursion assumes non-contentious overlap of copies and kernels.	Partly accepted	The recursion models a single CUDA stream, in which copies and kernels are serialised, not overlapped, so it does not assume concurrent overlap. Contention can arise only if copies and kernels run concurrently on different streams, or if kernels read host memory. v6 counts concurrent copy-kernel intervals in every trace; where any exist, a contention term is measured per pairing by a microbenchmark (copy bandwidth and kernel time while running together) and entered as data. A fitted penalty is rejected: it would be tuned on the outcomes it predicts.
Major: no learned baseline.	Accepted	A gradient-boosted tree predictor (LightGBM) trained on the same component features, with leave-one-pairing-out training. Features and hyperparameters are fixed at lock. If it beats the recursion on held-out pairings, that is reported as a finding against the structural model.
Driver serialisation: launch costs are not independent.	Accepted, made precise	The CUDA launch queue holds roughly a thousand pending launches; when it is full, a launch blocks until a slot frees. The recursion gains a queue term with depth Q measured per pairing. Host costs are measured as bursts of in-sequence null-kernel launches, not single launches.
NUMA node affinity not controlled.	Accepted	Topology recorded (GPU-to-CPU affinity); threads pinned to the GPU's local NUMA node; link microbenchmarks run from that node.
Pageable versus pinned host memory.	Accepted	Link microbenchmarks measure both; each copy in a trace is classified by memory kind and costed accordingly.
Unified or host-mapped memory couples device and host cost.	Accepted as a gate	The stack uses no managed memory. Any unified-memory page fault in a trace flags the row.

Ablation: precomputed oracle masks to separate estimator cost from synchronisation.	Accepted	A precomputed-mask arm: each estimator's masks are computed offline and preloaded on the device, then the same kernel runs. The difference from the live arm is the estimator's compute plus its synchronisations, split further by the trace.
Recall may hide which heads were captured (local versus retrieval).	Accepted as analysis	This is what H4 tests. v6 adds recall stratified by head type (retrieval-like versus local heads, by a criterion fixed at lock) to the H4 analysis.
Models of 70B parameters and above may break separability.	Declined	Those models need several GPUs and add interconnect effects outside this study. Stated as a boundary.
Use a serving simulator (APEX, LLMervingSim 2.0) as a baseline.	Declined as a baseline	The simulator called APEX here is Lin, Kwon, Pineda and Nina Paravecino (arXiv 2411.17651), a serving-configuration simulator that LLMervingSim 2.0 compares against. It is a different paper from the IPDPS 2026 APEX (Fan et al.) discussed above, and it has not been read. Both simulators take per-model, per-device operator profiles and target serving behaviour; neither predicts a single sparse prefill on an unprofiled pairing. They are positioned in related work. [Read in full after v7: the APEX simulator profiles operations per device, not per model (about 40 GPU hours per cluster type, 6 to 8 hours for a new GPU), and interpolates between profiled points. Its fidelity is reported as the error of predicted speedup between execution plans (10.7% on average); its predicted per-token time is consistently below the measured one, which the authors attribute to leaving out operations other than key ones such as attention and feed-forward. It has no host-side term. The decision not to use it as a baseline stands.]

Response to the fourth referee report

Referee point	Response	Change in v7
Major: link cost is bundled into d_i ("device plus link"), which contradicts the three-way split and makes device costs non-transferable across hosts (H0, H2).	Accepted as a presentation fault; the test was not confounded	The wording was inconsistent and the brief also dropped the synchronisation cost from the recursion. H0 and H2 were always defined on c^{dev} , measured by isolated replay, with c^{link} measured on every target pairing and never transferred, so no link cost entered a separability test. The recursion now carries the link term separately: $D_i = \max(H_i, D_{i-1}) + l_i + d_i$, with $d_i = c^{\text{dev}}_i$ only and $l_i = c^{\text{link}}_i$. Copies are operations with $d_i = 0$.
Major: CPU frequency cannot be fixed on cloud VMs; "same frequency band" is undefined and unverifiable; Elango et al. show a 300-fold stability change across 200 MHz.	Accepted, with a correction	The control is replaced. A guest often cannot read the true core clock, so the band is now defined by a measured quantity: a fixed single-thread CPU loop and a null-kernel launch burst, each timed at the start, middle and end of every session and between ground-truth blocks. Their distributions are reported for every session. A session is usable if the probe's coefficient of variation stays under a bound, and a component transfers (H2) only if source and target probe medians differ by less than a bound; both bounds are fixed at lock from the repeat-session noise floor on P1. Sessions failing either are rerun, not reweighted. Correction: the 300-fold figure is the coefficient of variation of 5G throughput in a single exploratory run of a pipeline tied to 0.5 ms slot deadlines; the validated means were 3.0% and 16.5%. It motivates the check but does not show that a deadline-free prefill behaves the same way.

Major: H4 ignores layer depth; PLA (IEEE TCAD 2026) shows K_{90} falling from 435 to 308 tokens between layers 1 and 32, so aggregate recall cannot be a sufficient proxy.	Reporting accepted; the inference rejected	H4 already matches recall per layer, not in aggregate, so an estimator cannot reach a matched value by keeping shallow-layer mass while losing deep-layer mass; the brief said "matched recall" without "per layer", which is now fixed. PLA's numbers are confirmed (PointLLM-7B, instruction-to-point attention, layers 1 and 32), but PLA tests no estimator at matched recall, so it does not contradict H4; it shows that concentration varies with depth, which is a reason to report by depth. Added: recall and the H4 estimator effect are reported by depth band (first, middle and last thirds of layers, fixed at lock), alongside the head-type split.
Host CPU mask building may contend with GPU DMA for host memory bandwidth, even without copy-kernel concurrency.	Accepted	Link microbenchmarks are also run with a concurrent CPU load sized like the largest mask build, and traces flag any copy in flight while host-side estimator work runs. Where both occur, the loaded bandwidth is used for that copy; the difference is reported, never fitted.
The launch-queue depth Q is treated as fixed; add a sensitivity ablation with Q in {4, 8, 16}.	Accepted in a different form	Q is measured per pairing and per session. The CUDA launch queue holds on the order of a thousand pending launches (to confirm by microbenchmark), so 4 to 16 is not its scale. The sensitivity analysis instead recomputes every committed prediction with Q at one quarter, one half, twice and four times the measured value, and with the queue term removed, and reports how much each prediction moves. The committed prediction uses the measured Q .
Cloud hypervisors are not stationary; noisy neighbours add variance to h_i .	Accepted	Components and ground truth already run in the same session, interleaved. The host-speed probe above now also detects drift within a session; the tenancy of each instance type (whole host or shared) is recorded; repeat sessions on P1 set the between-session floor.
The CTFusion-style baseline may underfit on six pairings without explicit bottleneck features.	Accepted	Bottleneck ratios were already included. Now listed explicitly: arithmetic intensity per operation class, host-to-device time ratio, transfer demand relative to measured link bandwidth, and synchronisation count relative to operation count.
$G1$ is a framing gap relative to CTFusion; $G2$ and $G3$ carry the novelty.	Accepted	$G2$ and $G3$ are now the lead contributions. $G1$ is kept as the validation question (does the structure help when training hardware is scarce), not as a claim of a new problem.
HyGIN suggests host mask building may be an implementation artefact that persistent GPU kernels remove.	Noted	The study predicts the cost of estimators as deployed, not that host work is unavoidable. Estimators whose selection runs entirely on the GPU are already included, and H5(b) predicts their crossover moves less. Persistent-kernel designs are out of scope.
LLMEgger (Shen and Mashayekhy, IEEE ICPEC 2026).	Read; judged distant	Read in full (DOI 10.1109/ICFEC69006.2026.00010). It schedules pipeline and tensor parallelism for FLAN-T5 on CPU-only edge pods (1 vCPU each, links capped at 1 Gbps) from offline-profiled lookup tables. Its "other overhead" category (launch and dispatch) appears in one breakdown figure but is not quantified in the text, so it is not cited as evidence about prefill overhead.

Papers read after v7

The researcher supplied five further papers on 4 Oct 2026, all read in full. None closes a gap. One wording in v7 was an overclaim (the oracle arm called a quality ceiling) and is corrected; one definition the study needed was missing (which forward pass recall is measured against) and is now fixed.

Paper	What the full text shows	Change
APEX simulator (Lin, Kwon, Pineda and Nina Paravecino; arXiv 2411.17651 v2; no venue in the PDF)	Serving simulator that searches parallel execution plans from per-device operation profiles, with linear interpolation between profiled points. Fidelity is the error of predicted speedup between plans (10.7% on average). Predicted per-token time is consistently below measured because operations outside the key ones are left out. No host-side term; the CPU only runs the simulator.	Baseline decision unchanged; its stated reason corrected (per-device, not per-model, profiles). Its systematic under-prediction from omitted operations supports G1's premise.
CompactAttention (Song, Jo, Kang and Kim; arXiv 2605.16839)	Under chunked prefill, block-sparse methods lose for two reasons: kernels lose efficiency when the query chunk is much shorter than the keys, and pattern search reruns over the accumulated keys at every chunk. On Llama-3.1-8B with TP = 2, XAttention's end-to-end chunked prefill is 0.68x to 0.73x of dense speed on H200 (chunk 1024) and 0.78x to 0.84x on RTX PRO 6000 (chunk 512), from 8K to 128K. Host CPUs are not reported. Accuracy is compared at operating points chosen per method. Union of block masks changes executed density.	The chunked-prefill extension now separates the two causes and reports mask density and executed density separately. CompactAttention's figures are an external reference, not used to set our committed predictions.
Oracle-guided sparse prefill (Wang, Razanajato, Yuan, Zhang and Liu; arXiv 2606.07703, technical report)	Head-averaged top-k attention-mass oracle, used as a controlled reference; the authors state it is not a global optimum. Per-query oracle within 1 point of dense on Qwen-family models; support shared by 64 queries falls from about 96 to about 80 at 128K when top-k drops to 256. Oracle decisions are made along the sparse forward pass. Recall of oracle support by their learned indexer is named as a diagnostic not yet run. Evaluation data and code are not released.	Oracle arm relabelled as a reference, not a ceiling. Recall reference fixed at lock (Quality measures). G3 risk added (Known weaknesses).
AB-Sparse (Liu, Wang, Chen, Gong et al.; arXiv 2605.12110)	Decode-side (Quest and ArkVale with per-head block sizes). Recall, the share of attention mass in the selected blocks, at block sizes 16, 32 and 64 with a fixed 4,096-token budget, differs by head and is stable across inputs (calibrated on Wikipedia, held on RULER). Accuracy gains accompany recall gains at a fixed budget; accuracy at matched recall is not tested.	K5 scoped as the prefill and block-128 extension of its measurement. Recall definition matches ours.
SKIP/TKLQT (Vellaisamy et al.; arXiv 2504.11750 v1; no venue in the PDF)	Three systems (AMD EPYC 7313 with A100-SXM4, two Xeon Platinum 8468V with H100 PCIe, GH200). TKLQT sums, over kernels, the time from the start of each launch call to the start of its kernel, so it mixes launch and queueing; it locates the CPU-bound to GPU-bound transition as batch size grows (512-token prefill, models up to 1.24B, eager PyTorch). Single-thread host speed dominates at small batches. Host and GPU change together across its systems. Null-kernel launch overhead 2.26, 2.37 and 2.77 μ s.	P4's null-kernel check now has two Sapphire Rapids references with different values, compared only under a matching definition. The recursion's queue term separates what TKLQT mixes.

The research question

Can the end-to-end speedup and the accuracy of training-free block-sparse prefill be predicted for host-GPU pairings on which the workload has not been run, from component measurements alone, and where does the prediction fail?

Primary claim and conditional claim. The primary claim concerns predicting end-to-end time at a given density. Profitability at non-inferior accuracy is reported only where a non-inferior density exists. On the pilot model that may be only the oracle at density 0.5, so profitability for deployable estimators may turn out to be empty, and that would be reported as such.

Why now. Kernel-launch latency barely moves across GPU generations (about 6.1 μ s on a V100, 6.0 on an A100 and 4.15 on an H100, IEEE LANMAN 2026) while attention kernels keep getting faster, so fixed host-side terms take a growing share of prefill on faster GPUs. Published reports of sparse methods running slower than dense, including under chunked prefill (CompactAttention), each come from one or two machines.

Terms used throughout:

- **Unmeasured pairing:** a host-GPU pairing on which the full workload has not been run when the prediction is written. Allowed inputs are (a) host-platform components measured on that host without running the workload on the GPU, (b) device components of that GPU, (c) link microbenchmarks on the pairing, taking seconds. The prediction file and its sha256 are written to the synced session log before the workload phase starts; the analysis refuses any prediction stamped later.
- **Host component c^{host}_i :** host time to issue operation i (framework dispatch, library front end, launch floor) plus any host-side compute (e.g. CPU mask building). Measured with device work replaced by null kernels, following TaxBreak's protocol, and by running host-side compute directly at target sizes. The null-kernel floor on P4 (Sapphire Rapids with H100) is checked against two published Sapphire Rapids figures: TaxBreak's floor on a DGX H100 with a Xeon 8480C (about 4.7 μ s) and SKIP's launch overhead on a Xeon Platinum 8468V with an H100 PCIe (2.37 μ s, from the start of the launch call to the start of the kernel). They differ by about a factor of two and need not measure the same interval, so P4's floor is reported with its exact definition and compared only with a figure measured the same way; otherwise the comparison is recorded as indicative. A single-thread CPU benchmark is recorded for every host, because eager dispatch runs on one thread (TaxBreak).
- **Device component c^{dev}_i :** GPU execution time of operation i , from isolated per-operation replay on the target GPU with the arm's recorded masks, never from a run of the composed workload.
- **Operation sequence and synchronisation map:** the ordered operations of each arm and the positions of its GPU→CPU synchronisations. They depend on the software, not the hardware, so they are taken from a P1 trace and checked afterwards against each pairing's kernel list.
- **Link component c^{link}_i :** cost of operation i that depends on the pairing: host↔device copies and synchronisation round trips. Measured per pairing by microbenchmarks.
- **Estimator, recall, accuracy, profitable:** as in v3. Recall is the share of the oracle's captured attention mass that kept blocks capture; accuracy uses a paired non-inferiority test (which one is open, Questions item 5); profitable means end-to-end speedup above 1 over the fastest correct dense kernel at non-inferior accuracy, estimator cost included.

What the earlier work established

Earlier oracle-mask study (three GPUs)

Finding	Evidence and caveat
Kernel speedup does not survive end-to-end	On the A100 at 0.75 sparsity the sparse kernel ran 1.28× (8192 tokens) and 1.96× (16384) faster than flash; end-to-end prefill with the reference mask builder was 0.75× and 0.63×.
The sign of the result depends on the card	At 32768 tokens and 0.75 sparsity: about 1.3× on an L4 and about 0.48× on an A100 with the reference builder.

The cause is host-side mask construction	Both hosts share a CPU platform; builder costs within about 3%. The L4's slower GPU hides the CPU work; the A100's does not. A vectorised builder turned the A100 result into a speedup at 16384 tokens.
The mechanism held on a third card	Of three predictions registered before the H100 run, two passed (exposed-share model 9 of 9 cells; faster CPU builds masks 2.1–2.4× faster) and one failed (7 of 9 cells).
The oracle is expensive	Scoring costs about 35× the latency the sparsity saves at 32768 tokens on an L4.
A cheap estimator trails the oracle	Mean-pool trails the oracle by 32–40 points at 1.5B and 39–76 at 7B on the harder retrieval task at 16384 tokens.

T4 pilots on the L4, Qwen2.5-1.5B-Instruct (1-3 Oct 2026)

Finding	Evidence and caveat
Even the oracle certifies only at density 0.5	Primary claim only on qa_1/16384 (70 vs 68 of 100, bound −9.0); secondary on niah_multivalue/16384 (bound −7.7). Nothing at 0.75 or 0.9.
Inline mean-pool certifies nowhere	7–24 points below dense at 0.5, bounds −22.2 to −42.0; estimator plus mask build cost 0.06–0.31 of the kernel saving.
Scalar-threshold XAttention certifies nowhere	At $\tau = 0.95$ bounds −12.7 to −50.8 at densities 0.11–0.39; niah_multivalue/32768 collapsed to 0 of 50. Dense reproduced 400 of 400 predictions.
Density depends on the prompt at fixed τ	33% of blocks on qa_1 versus 11% on NIAH essays at 32768.
Estimator cost depends on the code path	On the L4's torch fallback, xattn_estimate costs more than dense attention (19.5 vs 14.5 ms at 16384; 60.6 vs 58.4 ms at 32768).
Calibrated XAttention	Built and pre-registered; not run.

Caveats carried from v3: estimated A100 dense baselines; some 32768-token rows exceed the position limit; three mask-rule eras; four headline claims revised when a second point was added; the T4 pilots cover one model, one card and synthetic tasks.

Research gaps targeted

G1 to G3 are the contribution; K1 to K9 are controls. Positioning rests on Appendix A and must be re-checked in IEEE Xplore and Scopus before submission.

#	Gap	How the study addresses it	Nearest work
---	-----	----------------------------	--------------

G1	End-to-end latency on unseen CPU-GPU pairs can be predicted from host, transfer and device micro-benchmarks with a learned model (CTFusion, IEEE FLTA 2025), but for workloads whose host side is input preprocessing, with host-device interaction learned implicitly from 20 training systems. KernelSight-LM measures host constants per host and engine and assumes they do not depend on the GPU; when it predicts a new GPU it borrows them from the closest profiled system. That assumption is not tested there. Whether an explicit critical-path model of measured components predicts data-dependent sparse prefill, whose host side is dispatch, mask building and synchronisation inside the forward pass, better than a CTFusion-style learned predictor, especially when only a few hardware systems are available for training, has not been tested.	Component-composed recursion against a CTFusion-style learned predictor and two simple baselines, on seven physically distinct pairings including a held-out host platform and a full-transfer pairing; predictions committed before each pairing runs.	CTFusion, nn-Meter, HELP, CloserToMe, KernelSight-LM
G2	Host-overhead summaries (HDBI, TKLQT) are ratios or sums, and KernelSight-LM uses one maximum per step plus a serial tail, so it counts host work but not where it falls. Serving simulators (LLMServingSim 2.0) model dependencies and synchronisation between devices from per-device profiles. TaxBreak states that its replay-based attribution can be imperfect for synchronisation-heavy kernels. No lightweight model represents data-dependent mask generation and its GPU-to-CPU synchronisations inside a prefill step, from measured components.	Per-operation recursion with synchronisation points, launch-queue depth and measured contention; tested against sum, per-step maximum and learned baselines (H1); oracle-mask ablation.	TaxBreak, SKIP/TKLQT, KernelSight-LM, LLMServingSim 2.0
G3	Whether attention recall is a sufficient proxy for accuracy across estimators is untested. The Sparse Frontier lists as unknown how close a recall-maximising mask is to a task-optimal one. Less Is More treats recall as necessary but not sufficient in long reasoning decodes and compares recall across methods on one problem. VSPrefill shows accuracy rising with recall on one dataset (HotPotQA), without stating which selection method produced the recall levels, and compares recall, not accuracy at matched recall, across selection methods. Oracle-guided sparse prefill names recall of oracle support by a learned indexer as a diagnostic not yet run. AB-Sparse uses recall to choose block sizes for decoding and reports accuracy gains at a fixed token budget, not at matched recall. CompactAttention compares accuracy at operating points chosen per method.	Paired test at matched recall across oracle, mean-pool, XAttention and (where available) DuoAttention-style masks, at block sizes 32, 64 and 128 (H4).	VSPrefill, The Sparse Frontier, Less Is More, oracle-guided sparse prefill, AB-Sparse

#	Control	How	Status
K1	Implementation quality confounds estimator comparisons	One per-head selector and one block-sparse kernel for every arm.	Built, CPU-tested
K2	Weak dense baselines	Fastest correct dense kernel per card and band; FA3 on the H100.	Drafted
K3	Calibration leakage	Disjoint splits with a pre-run gate.	Built
K4	One model family	Qwen2.5-1.5B, Qwen2.5-7B, Llama-3.1-8B (held out of all fitting).	Planned
K5	Block-size dilution	Recall at block sizes 16–128 at matched density. AB-Sparse already reports, for decoding, that recall against block size (16 to 64, at a fixed 4,096-token budget) differs by head and is stable across inputs, so this is a control only; K5 extends it to prefill and to block 128.	Drafted
K6	Measurement integrity	Pre-registration, checks that fail on purpose, provenance on every row.	In place
K7	Non-inferiority at a floor	Cells count only if dense accuracy $\geq$ a pre-registered floor.	Not built
K8	Code-path differences	Code path recorded per row; fallbacks never on the same curve.	Built
K9	Predictions fitted after the fact	Every prediction written, hashed and synced before the workload it predicts runs (formerly G4).	Not built

Hypotheses

Each has a statistic and a tolerance fixed at lock. Failures are reported as failures. Results are reported per pairing; nothing is pooled into a significance test across pairings.

#	Hypothesis	Test and falsifier
H0	Separability. Host-platform components measured with one GPU equal those measured with another GPU on the same host platform; device components measured on one host equal those on another host for the same GPU. Link components are pairing-specific.	Host side: P1 vs P2 (Cascade Lake) and P3 vs P5 vs P6 (AMD EPYC 3rd gen). Device side: P1 vs P3 (L4) and P4 vs P6 (H100). Per-operation differences within <i>[set before lock]</i> , after allowing for the provider-level variance measured with the same dense workload on both providers. The host side tests the assumption KernelSight-LM makes without testing it: that host constants do not depend on the GPU. If H0 fails, H2 is not tested and the scope narrows to components measured on the target pairing.
H1	Composition. On every pairing, the recursion over measured components predicts untraced end-to-end prefill time within tolerance, and on held-out pairings predicts it better than an overlap-free sum, the KernelSight-style step form and a learned predictor trained on the same components. Stated in advance: on dense arms, which have no synchronisation inside the step, the recursion and the KernelSight-style form should agree (KernelSight-LM reports about 2% composition error when per-step times are measured). Its advantage is predicted only on arms with data-dependent synchronisations.	Per pairing: absolute relative error within <i>[set before lock]</i> ; sparse-vs-dense sign correct outside the resolution floor; recursion error below all three baselines on held-out pairings, reported separately for dense arms and for arms with synchronisations. A large advantage on dense arms would point to an error in the baseline, not support for G2. If the learned predictor wins, that is reported as evidence against the structural model.

H2	Transfer. Predictions stay within tolerance when components come from other pairings: host components for P2 (from P1) and P5 (from P3); device components for P3 (from P1); and both for P6, whose host components come from P3 and P5 and whose device components come from P4.	Same statistic as H1. P6 is the full-transfer test: only its link microbenchmarks are measured on it before its prediction is committed. Conditional on H0.
H3	Forecast. On P4 (H100 with Sapphire Rapids, a host platform used nowhere else), the crossover context length per estimator falls inside the band predicted before P4's workload runs.	Predicted band versus measured crossover with bootstrap intervals.
H4	Recall sufficiency. At matched per-layer recall, accuracy does not depend on which estimator chose the blocks, at block sizes 32, 64 and 128.	Paired comparison at matched per-layer recall, or the estimator coefficient in a pre-registered model, against an equivalence bound. Also reported by depth band (first, middle and last thirds of layers) and by head type. Either outcome reported.
H5	Direction of the crossover (secondary). (a) On a fixed host platform, a faster GPU moves the crossover to longer contexts for estimators with host-side steps (AMD EPYC: L4, then L40S, then H100). (b) For an estimator whose selection runs entirely on the device, that shift is smaller. (c) On a fixed GPU, the host with the lower measured host-component cost gives the shorter crossover (L4: P1 vs P3; H100: P4 vs P6).	Signs predicted from measured component ratios before data, with the tolerance stated on the input ratio, as learned from the earlier H100 test. These are consequences of H1, reported separately.

Methodology

Hardware pairings

GCP fixes the host CPU platform by machine series: the L4 (G2) and the A100 (A2) run on Intel Cascade Lake, and the H100 (A3) on Intel Sapphire Rapids. AWS pairs the L4 (G6), the L40S (G6e) and a single H100 (p5.4xlarge, AMD EPYC 7R13) with third-generation AMD EPYC, and the A10G (G5) with second-generation AMD EPYC. That gives a crossed design:

Pairing	Host platform	GPU	Provider	Role
P1	Intel Cascade Lake	L4	GCP G2	Anchor; source of transferred components.
P2	Intel Cascade Lake	A100	GCP A2	Second GPU on Cascade Lake: host-side H0, host transfer (H2).
P3	AMD EPYC 3rd gen	L4	AWS G6	Same GPU as P1 on a different host: device-side H0, device transfer (H2).
P5	AMD EPYC 3rd gen	L40S	AWS G6e	Second GPU on the AMD host: host-side H0, host transfer (H2), GPU-speed axis (H5a).
P4	Intel Sapphire Rapids	H100	GCP A3	Host platform used nowhere else: forecast (H3).
P6	AMD EPYC 3rd gen	H100	AWS p5.4xlarge	Same GPU as P4 on a different host: device-side H0; full transfer (H2); H5a and H5c. Run last.
P7 (optional)	AMD EPYC 2nd gen	A10G	AWS G5	Another host generation and a slower Ampere GPU; adds a held-out point if time allows.

Pairings are numbered by role; the run order is P1, P2, P3, P5, P4, P6, then optionally P7, so that every transferred component exists before the pairing that uses it. "Same platform" is not the same SKU: the exact CPU model string, core count, SMT state and observed frequency are recorded on every row and

compared before H0 is read. AWS service quotas for G and P instances must be granted before launch, and regional availability is checked at that point.

Per-pairing session protocol

1. **Environment.** One container image across providers (same CUDA runtime, PyTorch and kernel builds). Driver version matched where the provider allows and recorded otherwise. Threads pinned; CPU frequency sampled through the session; clocks locked where permitted and recorded.
2. **Link microbenchmarks** (seconds): host↔device bandwidth both ways, 1-byte copy, synchronisation round trip, PCIe generation and width.
3. **Host components:** the workload's operation sequence with device work replaced by null kernels (dispatch and launch costs); CPU-side compute (e.g. the reference mask builder) run directly at target sizes.
4. **Device components:** every operation of every arm replayed in isolation under Nsight Systems, with masks and shapes recorded from P1; per-operation GPU durations. Data-dependent work (e.g. the sparse kernel at a prompt's realised density) is replayed with that prompt's recorded mask. Graph replay of capturable arms as a cross-check on the summed device time.
5. **Prediction committed:** predictions for this pairing computed from steps 2–4 (and, for H2, from transferred components), written with their sha256 to the session log and synced before step 6.
6. **Ground truth:** untraced end-to-end prefill per arm, band and batch, arms interleaved, with a dense control.
7. **Profiler overhead:** traced versus untraced end-to-end time, reported per arm.

Cost model

For one stream, operation i has host cost $h_i = c^{\text{host}}_i$, device cost $d_i = c^{\text{dev}}_i$ (GPU execution only) and link cost $l_i = c^{\text{link}}_i$ (host-device copy time; zero for a kernel). Device costs are the only ones transferred between hosts (H2); link costs are always measured on the target pairing. With H_i the time the host has issued operation i and D_i the time the device finishes it:

$H_i = H_{i-1} + h_i$
 if Q launches are pending: $H_i = \max(H_i, D_{i-Q})$ (finite launch queue, Q measured per pairing)
 at a synchronisation point: $H_i = \max(H_i, D_{i-1}) + c^{\text{link, sync}}$
 $D_i = \max(H_i, D_{i-1}) + l_i + d_i$ (+ measured contention, only where a trace shows a concurrent copy)
 end-to-end prefill time = D_N

Scope of the execution model. One CUDA stream: copies and kernels on it are serialised, so the model does not assume that they overlap. Every trace is checked for copy-kernel concurrency and unified-memory faults; contention is measured, never fitted.

Baselines from the same components: sum, $\Sigma h + \Sigma l + \Sigma d$ (no overlap); **KernelSight-style step form**, $\max(\Sigma h, \Sigma l + \Sigma d)$ per layer plus every synchronisation cost added as a serial tail (KernelSight-LM's overlap floor and serial tail, rebuilt from our components); **learned**, a CTFusion-style LightGBM predictor: task-demand features (context length, density, batch, operation and synchronisation counts), hardware micro-benchmark features (host, link and device costs, including pinned and pageable transfer bandwidth) and bottleneck ratios (arithmetic intensity per operation class, host-to-device time ratio, transfer demand relative to measured link bandwidth, synchronisation count relative to operation count), trained leave-one-pairing-out with features and hyperparameters fixed at lock. The recursion uses the same synchronisation costs as the KernelSight-style form and differs from it only through operation order, synchronisation positions and the queue term, so the gap between the two isolates G2; the learned baseline tests whether that structure adds anything over regression (G1). HDBI is reported as a diagnostic.

Batch. Batch 1 and 4 at 16384 on the L4 pairings; batch 4 at 32768 where memory allows on the A100 and H100. Host cost per launch is roughly fixed while device work grows with batch, so the model predicts where host exposure vanishes; the predictions are tested like any other.

Crossover. Sparse is profitable where $D_N(\text{sparse}) < D_N(\text{dense})$. Solving for context length gives each estimator's crossover per pairing (H3 on P4).

Chunked prefill (extension). On one pairing (P5) with two estimators (inline mean-pool and calibrated XAttention), prefill is also run in chunks of 2048 and 4096 tokens at 16384 and 32768. Each chunk reruns the estimator over the accumulated keys, which the recursion represents as repeated operations; the predicted re-selection overhead is committed before the run. Whether the block-sparse kernel supports query chunks shorter than the key length is checked first; if it does not, the extension is dropped and that is recorded. CompactAttention attributes block-sparse losses under chunked prefill to two causes, the kernel's loss of efficiency when the query chunk is much shorter than the keys and the per-chunk rerun of pattern search, and reports XAttention at 0.68× to 0.84× of dense speed end to end on two GPUs. The extension therefore records the kernel's measured time at each query-to-key ratio separately from the per-chunk estimator operations, and reports mask density and executed density separately. CompactAttention's figures are an external reference only; the committed predictions come from the recursion.

Delay injection. Optional, as a check that the timing instrument registers a known host delay. Never cited as evidence for H0-H3.

Confounders and controls

Confounder	Control
CPU microarchitecture, caches, memory bandwidth	Inside c^{host} , which is measured on the target host at target sizes; never synthesised.
PCIe generation, link bandwidth, copy and sync latency	c^{link} measured per pairing by microbenchmarks; PCIe generation and width recorded.
Dynamic frequency scaling, boost clocks	Frequency sampled throughout; warm-up before measurement; components and ground truth measured in the same session, interleaved; clock state recorded per row. Cloud VMs may hide governor control and the true core clock. CPU frequency can interact with CPU-GPU timing non-monotonically (Elango et al., IEEE INFOCOM 2026, on an A100 server). Control: a timed host-speed probe (fixed single-thread loop and null-kernel launch burst) at the start, middle and end of every session and between ground-truth blocks; its distribution is reported per session; a session is usable only if the probe's coefficient of variation is under a bound, and a component transfers (H2) only if source and target probe medians differ by less than a bound. Both bounds are fixed at lock from the P1 repeat-session floor. Failing sessions are rerun.
vCPU topology, SMT, noisy neighbours	Thread pinning; topology and instance tenancy recorded; host-speed probe detects drift within a session; repeat sessions on P1 for between-session floors.
Host-memory contention between CPU mask building and DMA	Link bandwidth also measured under a concurrent CPU load sized like the largest mask build; traces flag copies in flight during host-side estimator work, which are then costed at the loaded bandwidth; the difference is reported.
Driver and software differences across providers	One container image; driver recorded and matched where possible; dense control every session; H0 detects a residual device-side difference.
Measurement asymmetry between arms	One device-time method (isolated per-operation replay) for all arms; graph replay as a cross-check only.
Prediction made trivial by measuring the workload	No input to a prediction comes from a run of the composed workload on the predicted pairing; the operation sequence comes from P1 and is verified afterwards.
Profiler overhead	Ground truth from untraced runs; traced-versus-untraced difference reported.

Prediction fitted to results	K9: predictions hashed and synced before the predicted workload runs.
NUMA placement	GPU-to-CPU affinity recorded; threads pinned to the GPU's local node; link microbenchmarks run from that node.
Pageable versus pinned host memory	Both measured; each traced copy classified by memory kind and costed accordingly.
Launch-queue backpressure	Queue depth measured per pairing; host costs measured in bursts.
Unified or host-mapped memory	Not used; any unified-memory fault in a trace flags the row.
Copy-kernel contention	Concurrency counted in every trace; measured by microbenchmark where present.

Arms

Arm	What it does	Role
Oracle	Pools full dense-softmax attention mass to blocks, from a separate dense pass (one ranking per KV head, meaned over its query heads).	Attention-mass reference, highest cost. Not a guaranteed quality ceiling: the mask keeping the most attention mass need not be best for the task (oracle-guided sparse prefill; The Sparse Frontier). Corrected after v7.
Window (sink plus local)	First block, diagonal and nearest earlier blocks.	Zero-cost floor
Mean-pool	Pooled query and key blocks (MInference-style), per head; inline device builder.	Cheap deployable baseline
Vertical-slash-style	Vertical and slash scoring reduced to blocks.	Different estimator structure
Calibrated-threshold (XAttention)	Antidiagonal scoring with per-(layer, head) thresholds from the authors' profiler, run as shipped including its GPU-CPU syncs.	Referee-expected baseline; data-dependent density and syncs
DuoAttention-style (conditional)	Retrieval heads dense, streaming heads sink plus local, from the authors' released head patterns; only for a study model with released patterns; licence checked first.	Head-differentiated recall profile for H4; calibrated, not training-free
Dense	Fastest correct kernel per card and band.	Reference
Precomputed-mask (ablation)	Each estimator's masks computed offline and preloaded on the device; then the same kernel runs.	Separates estimator compute and synchronisation from kernel and launch cost

All arms pass through one per-head selector and one block-sparse kernel call. A bitwise gate requires the selector to reproduce the earlier head-uniform rule exactly when heads share scores. The arms also supply what the cost model must explain: different host work, device work and synchronisation patterns.

Quality measures

- **Intrinsic recall:** budgets 0.50, 0.25, 0.10 and 0.05; null scorer as a floor; budget selection on 32 examples per task.
- **Recall on accuracy rows:** recall computed on the very masks that produced each accuracy row, per layer.

- **Recall reference (fixed at lock):** for every arm, recall at a layer is the share of that layer's dense attention mass, computed from its actual inputs in the arm's own forward pass, that the kept blocks capture (the convention oracle-guided sparse prefill uses for its oracle). Recall against the separate dense pass, which the oracle arm uses to choose its masks and whose scores are already cached, is reported as a secondary measure. The two differ because each sparse layer changes the next layer's inputs, so an arm can score well on one and badly on the other; H4 is read on the primary one.
- **H4 across block sizes:** on the primary cell (qa_1/16384, n = 100), accuracy at block sizes 32, 64 and 128 through a reference masked-attention path that is untimed and used for quality only; the fast kernel stays at 128. Block 16 only if the reference path supports it.
- **Extrinsic accuracy:** paired non-inferiority (test to be fixed); the dense arm must reproduce exactly in every run.
- **Where quality is measured:** once, on the primary GPU, with the dense control replicated on a second GPU.
- **Dense-accuracy floor (K7):** a cell counts only if dense accuracy clears it.
- **Recall by head type and by depth:** for H4, recall is also reported separately for retrieval-like and local heads (criterion fixed at lock) and for the first, middle and last thirds of layers, since attention concentrates with depth (PLA, IEEE TCAD 2026), so matched recall cannot hide different coverage of heads or layers.
- **Critical density:** the lowest oracle density still non-inferior; descriptive only.

Implementation status

Component	Status	Note
Per-head selector, bitwise gate, mask-rule registry	Built, CPU-tested	
Recall pass; null scorer; budget selection	Built	GPU check at 16384 not run.
Inline mean-pool arm; scalar XAttention arm	Built, run (L4)	T4 pilots.
Calibrated XAttention	Built, not run	Sessions A and B pre-registered.
Exact paired non-inferiority analysis	Built, run	
GCP launch, preflight, budget gate, teardown guards	Built, used	Teardown refuses a wrong zone.
AWS launch, cleanup, budget cap, hard-delete limit, teardown	Not built	For P3, P5, P6 and P7 (G6, G6e, p5.4xlarge, G5). Must meet the same guarantees as the GCP scripts and be tested against a fake CLI before use. Service quotas requested in advance.
Chunked-prefill path	Not built	Extension on P5; kernel support for short query chunks to check first.
Learned baseline (LightGBM) and leave-one-pairing-out training	Not built	New in v6; features fixed at lock.
Launch-queue depth, NUMA, pinned/pageable and contention microbenchmarks; trace checks for concurrency and unified-memory faults	Not built	New in v6.
Precomputed-mask arm	Not built	New in v6; reuses recorded masks.
Common container image across providers	Not built	New in v4.
Link microbenchmarks	Partly built	Launch, sync and 1-byte copy floors exist; bandwidth and PCIe probes new.

Null-kernel host-component profiler and sync map	Not built	TaxBreak-style.
Isolated per-operation device replay with recorded masks; graph-replay cross-check	Not built	TaxBreak-style Phase 2.
Cost model, baselines (sum, per-step max) and HDBI diagnostic	Not built	
Prediction hashing and pre-workload sync (K9)	Not built	
Reference masked-attention path at block sizes 32 and 64	Not built	Quality only.
DuoAttention-style arm	Not built	Conditional on released patterns and licence.
FA3 build; Llama staging; first A100 session; H100 fault probe	Not built / not run	
Two adversarial reads; lock; related-work section	Not done	

Decision order

1. Run calibrated XAttention sessions A and B as pre-registered.
2. Build the component profilers, the model and its baselines; test H1 on P1. If the recursion does not beat the sum and per-step-maximum baselines on P1, stop and revise before any new pairing.
3. Test H4 on banked T4 rows plus recall; build the reference path for block sizes 32 and 64.
4. Cost and run the real-text haystack swap on the primary model.
5. Build and test the AWS tooling; request quotas; cost every pairing; fix all tolerances; lock.
6. Run P2, P3 and P5 (H0, H1, H2, H5), then P4 (H1, H3), then P6 (H0, H1, full transfer, H5), each with predictions committed first; the chunked-prefill extension on P5; P7 if time allows.

Known weaknesses and open risks

1. **Seven pairings is still a small panel.** Results are per pairing; the claim is limited to the measured host platforms and GPUs.
2. **Separability may fail.** If H0 fails, the model still composes components on each pairing (H1), but transfer (H2) is not tested and the scope narrows.
3. **Component profiling may misstate composed behaviour.** Null kernels can understate host cost, and isolated replay removes cache and queue interactions between neighbouring operations. TaxBreak itself notes that replay can be imperfect for dynamic, autotuned or synchronisation-heavy kernels, which describes our sparse arms. Before H1 is read, replayed kernel durations and kernel names are compared with the same kernels in the P1 trace, per operation family; mismatched kernel variants are flagged. H1's error on real pairings measures how much the rest matters.
4. **Profitability may be empty for deployable estimators.** In the pilots no deployable estimator was non-inferior at any tested density. The primary claim is therefore time prediction at a given density.
5. **Two providers add operational risk.** AWS tooling must meet the same cleanup, cap and teardown guarantees before any launch, and quotas or regional capacity can delay sessions.
6. **Cloud hosts are not controllable.** Governor and turbo state may be hidden; this is recorded, not assumed away. Because CPU frequency can change CPU-GPU timing non-monotonically, a frequency difference between sessions is a candidate explanation for any H0 failure and is checked first.
7. **Positioning.** CTFusion, nn-Meter, HELP, CloserToMe, KernelSight-LM, LLMservingSim 2.0, both APEX papers and TaxBreak narrow G1 and G2; the novelty rests on data-dependent synchronisation

inside a step and on validation across real pairings.

8. **Sparse speedups are small on this model.** Even the oracle arm's masks give modest speedups; the claims concern prediction. (Wording corrected after v7: the oracle arm is a reference, not a ceiling.)
9. **G3 may be partly pre-empted.** Oracle-guided sparse prefill names recall of oracle support by its learned indexer as its next diagnostic. That is one learned selector measured against its own teacher; H4 compares several training-free estimators at matched per-layer recall and asks whether accuracy depends on which one chose the blocks. If their follow-up appears first, H4 is positioned as the cross-estimator test.
10. **H4 may fail.** That is a result.
11. **The learned baseline may win.** CTFusion trained on 20 systems; ours has at most six training pairings per held-out test, so it may underfit or overfit; both outcomes are reported, and the comparison is made only on held-out pairings.
12. **Accuracy evidence is narrow;** block size fixed at 128 for timing; Qwen2.5-1.5B stops at 32768.
13. **Budget.** The researcher extended the budget on 4 Oct 2026 to cover the AWS pairings. Each session is still costed, capped and given a hard-delete limit before it launches.

Deliberately out of scope

- Models that need more than one GPU (70B parameters and above), and their interconnect effects.
- Decode sparsity; natively trainable sparse attention; continuous batching and batch sizes above 4; chunked prefill beyond the scoped extension.
- Synthetic host speeds as evidence.
- A new estimator as a headline; a coverage-guaranteed selector remains a possible extension.
- A timed kernel at block sizes below 128; cross-attention; linear attention; context above 65K tokens; families beyond Qwen2.5 and Llama-3.1.

Considered and rejected

Candidate	Why rejected
A head-adaptive or entropy-driven estimator as a contribution	Already explored: FlexPrefill (per-head patterns and budgets), Twilight (adaptive top-p budgets), AB-Sparse (per-head block sizes). Task-KV targets decode-side KV memory.
Attention recall by block size, per head	AB-Sparse already reports it. Kept as control K5.
Estimator cost against quality, benchmarked	Method papers already report estimator overhead (MInference, SALE) and serving papers report it under chunked prefill (CompactAttention).
Oracle masks as a quality ceiling	Oracle-guided sparse prefill (arXiv 2606.07703, read in full) uses a head-averaged attention-mass oracle as a controlled reference and states that it is not a global optimum; with support shared by 64 queries it falls from about 96 to about 80 at 128K when top-k drops to 256. Kept as an arm and a reference, not as a ceiling or a claim.
CUDA-graph compatibility as a standalone study	Dense CUDA-graph behaviour is characterised (IEEE LANMAN 2026). Capturability is recorded per arm instead.
An analytical sparse-kernel efficiency model carried across GPU generations	KernelSight-LM finds attention-kernel efficiency is device-specific and does not transfer across GPU generations; device components are measured instead. (SparkV, cited here earlier, has been withdrawn.)
A fitted contention penalty in the execution model	It would be tuned on the outcomes it predicts. Contention is detected in traces and measured instead.
A launch-queue sensitivity sweep over Q in {4, 8, 16}	Far below the measured scale of the CUDA launch queue; the sweep is made relative to the measured Q instead.

Treating PLA's depth-wise attention concentration as evidence against H4	PLA tests no estimator at matched recall, and H4 matches recall per layer; depth is reported as a stratum instead.
A serving simulator as the baseline	The APEX simulator (Lin et al., arXiv 2411.17651) uses per-device operation profiles and has no host-side term; LLMervingSim 2.0 uses per-model, per-device profiles; both target serving behaviour. The learned predictor and the KernelSight-style step form are the fairer comparators. (Corrected after reading the APEX simulator: its profiles are per device, not per model.)
Synthetic host speeds (delay injection) as evidence	Rejected by the v3 referee; retained only as an instrument check.

Questions for the reviewer

1. Does the crossed seven-pairing design (two GPUs on Cascade Lake, three on one AMD host platform, the L4 and the H100 each on two hosts, one host platform held out) answer the validation objection?
2. Is "workload not yet run on the pairing; components and link microbenchmarks allowed" an acceptable definition of an unmeasured pairing, and is P6's full transfer a convincing test?
3. Are the sum, per-step-maximum and learned (LightGBM) baselines the right comparators, and is leave-one-pairing-out training on seven pairings a fair test for the learned one?
4. Is predicting sparse-prefill time across real host-GPU pairings, validated out of sample, a sufficient contribution for an IEEE Transactions journal, or does it also need a design contribution, such as a selector built from the model's predictions?
5. If the model fails out of sample, is a well-characterised negative result publishable at the target venue?
6. Is H4 at block sizes 32-128 through an untimed reference path sufficient?
7. Which journal fits this scope (IEEE TC, TPDS, TCAD or ACM TACO), and is ISPASS or MLSys a better first target?

Data splits, governance and analysis

Split	Use	Rule
Calibration	Calibrated thresholds	Disjoint from evaluation by example and question index.
Selection	Budgets (32 examples per task)	Never reused for reporting.
Components	Model inputs per pairing	Measured before the pairing's workload; predictions hashed before ground truth.
Evaluation	Accuracy, cost, ground-truth timing	Ids disjoint from every banked id, enforced by a test.
Replication	Earlier pilot ids	Replication checks only.

Governance. Dated amendments before any data; two independent adversarial reads; tagged lock; calibrated XAttention first; P1 as the go/no-go gate; a hard cost cap, reservation check and fixed cut order that never removes ground-truth timing, H1 or H4; provenance (commit, mask-rule era, host CPU string, PCIe link, driver, library versions, clock state) on every row; no check trusted until it has failed on purpose; third-party code installed and called, never copied.

Planned analysis. Sensitivity of every committed prediction to the launch-queue depth (one quarter, one half, twice and four times the measured Q, and no queue term). Host-speed probe distributions per session. Per pairing: predicted versus measured end-to-end time for the recursion and the baselines, with residuals by arm and synchronisation count; H0 component differences; H2 transfer errors; the P4 crossover forecast against measurement; HDBI as a diagnostic. Quality: accuracy against matched

recall by estimator and block size. A record of what ran and what was cut, and a pre-registered statement of what the paper claims if only the never-cut core runs (P1, P2 and P3; H0, H1 and H4).

Appendix A. Novelty search log (4 Oct 2026, extended for v4 and v5)

Web search, one query per run, as specified by the researcher, followed by targeted look-ups of the nearest hits. A general web search under-indexes IEEE Xplore; this log is a starting point for a database search (IEEE Xplore, ACM DL, Scopus), not a substitute for it. Venues marked "to verify" were not confirmed from a primary source.

Group 1: kernel speedup versus system performance (G1, G2, K2)

Query	What came back
attention kernel speedup end-to-end latency	Sparse-attention method papers reporting kernel and end-to-end speedups on their own platform (FSA, Fused3S, FuseMax at MICRO 2024). None models why kernel speedup fails to carry over.
GPU microbenchmark deployed system discrepancy	GPU microarchitecture microbenchmarking (Wong et al., ISPASS 2010; Mei and Chu, IEEE TPDS 2017). Characterises hardware, not deployed-system gaps.
host CPU overhead GPU inference	DNN server overheads for vision (arXiv 2403.12981); vendor and blog material on host overhead. No sparse-attention case.
CPU GPU synchronization overhead deep learning	Mobile CPU-GPU co-execution synchronisation; multi-tenant interference on CPU-GPU sync. Different regime.
cross-generation GPU benchmarking machine learning	Hardware-assisted cross-generation prediction of GPUs under design (IEEE TCAD 2019): predicts next-generation GPU performance from current-generation statistics, 7.45–8.91% error. Closest IEEE precedent for prediction across generations; it targets GPU design, not host-device pairing. iMLBench (IEEE TPDS 2021): CPU-GPU integrated architectures.
kernel-level speedup wall-clock discrepancy	Thin; OS timekeeping threads only. The concept lives under "framework tax" and "launch overhead" instead.
Follow-ups	TaxBreak (IEEE ISPASS 2026, DOI 10.1109/ISPASS69572.2026.00014, read in full: two platforms, DGX H100 with Xeon 8480C and H200 NVL with Xeon Gold 6538Y+; isolation replay; null-kernel floor 4.7 μ s and 4.5 μ s; attributes a 10 to 29% drop in host orchestration to the faster host CPU, a comparison in which the GPU also differs); SKIP/TKLQT on CPU-GPU coupled systems (arXiv 2504.11750, venue to verify) [read in full after v7: no venue in the PDF; three systems, AMD EPYC 7313 with A100-SXM4, two Xeon Platinum 8468V with H100 PCIe, and GH200; TKLQT sums launch-to-kernel-start time over kernels, mixing launch and queueing; 512-token prefill, models up to 1.24B, eager PyTorch; host and GPU change together across systems; null-kernel launch overhead 2.26, 2.37 and 2.77 μ s]; Framework Tax (EMNLP 2023, not IEEE); Vidur (MLSys 2024, not IEEE); CUDA Graphs for distributed LLM inference (IEEE LANMAN 2026).
Added for v4 (referee citation)	APEX (Fan, Zhang, Li and Nikolopoulos, Virginia Tech; IEEE IPDPS 2026, DOI 10.1109/IPDPS65963.2026.00099, confirmed from the full paper supplied by the researcher): profiling-informed prediction of CPU and GPU subtask times to overlap CPU-offloaded attention with GPU execution in decode, on T4 and A10. Shows that predicting CPU-GPU overlap is studied; it does not predict end-to-end prefill on unmeasured pairings.
Added for v4 (pairing design)	Cloud host platforms: GCP G2 (L4) and A2 (A100) on Intel Cascade Lake; GCP A3 (H100) on Intel Sapphire Rapids; AWS G6 (L4) on third-generation AMD EPYC (AWS announcement, April 2024). To re-check against provider documentation at launch.

Added for v6 (supplied by the researcher, read in full)	Kohli et al. , IEEE HPEC 2025 (DOI 10.1109/HPEC67600.2025.11196674): CPU, integrated GPU and NPU on one AMD Ryzen AI chip; time to first token and per-token time for 1B to 3.8B models. Characterisation of one integrated platform; no prediction, no discrete PCIe pairing, no sparse attention. Distant. HyGIN (Huang and Kim, IEEE Computer Architecture Letters 2026, DOI 10.1109/LCA.2026.3711652): CPU- versus GPU-initiated RDMA writes on H100; the CPU path pays launch and synchronisation overhead, the crossover is platform-specific and calibrated offline. Supports measuring synchronisation cost per pairing; multi-node communication, not prefill. Distant. Elango et al. , IEEE INFOCOM 2026 (DOI 10.1109/INFOCOM59046.2026.11571401): on a GPU-accelerated 5G stack (A100), throughput stability is non-monotonic in CPU frequency, attributed to timing alignment between CPU scheduling, PCIe copies and kernels. Not a competitor; added as a confounder reference for the frequency control and the separability risk. PLA (Zhang et al., IEEE TCAD 2026, DOI 10.1109/TCAD.2026.3739022): accelerator co-design with top-k token pruning in prefill for point-cloud LLMs, evaluated in simulation against other accelerators. New hardware and a different modality; distant from all three gaps.
Added: analytical model CPU kernel launch GPU execution overlap critical path predict end-to-end latency deep learning inference	KernelSight-LM (Yao et al., Boston University and AWS; arXiv 2606.28565 v2, July 2026; read in full): roofline kernel model with a learned efficiency term, a host-overhead model and a discrete-event scheduler built on Vidur. Step time is an overlap floor that appears only when host work exceeds GPU time, plus a serial tail; host constants are measured per host CPU, GPU and engine but assumed independent of the GPU. Held-out GB200 from A100, H200-NVL and L40S; attention efficiency does not transfer across GPU generations (spread up to 3.8×), and its worst TTFT errors are long prefills of 19K to 23K tokens. Dense attention only. The closest threat to G1 and G2. Also: CPU-induced slowdowns in multi-GPU LLM inference (Chung et al., Georgia Tech, arXiv 2603.22774); MPK mega-kernels (arXiv 2512.22219).

Group 2: estimator cost/quality frontier (G3, G4, K1, K5)

Query	What came back
sparse attention overhead measurement	SALE (low-bit estimation for sparse prefill, arXiv 2505.24179), SpargeAttention, Rectified Sparse Attention: estimation overhead decomposed per kernel, usually excluding CPU-side scheduling.
block sparse attention GPU implementation	cuDNN Block Sparse Attention docs; Block Sparse Flash Attention (arXiv 2512.07011); FlashMoBA. Kernels, not estimator studies.
attention approximation accuracy latency tradeoff	A ³ (HPCA 2020, IEEE); vAttention, verified sparse attention with (ϵ , δ) guarantees (arXiv 2510.05688); Monte-Carlo attention (AAAI 2022).
long context inference acceleration benchmark	Minference (NeurIPS 2024), SampleAttention, Squeezed Attention, CoSA, SparseAccelerate (mid-range GPUs), Double-P.
importance estimation cost sparse transformer	The Sparse Frontier (Nawrot et al.; arXiv 2504.17768 v3, June 2026; read in full; ACL Findings 2026 per web search, not shown in the PDF): fine-grained per-query estimation in prefill is impractical because of estimation cost and kernel support. It reports FLOPs and memory traffic, not wall-clock time (its third limitation), and states as unknown how close a recall-maximising mask is to a task-optimal one (its sixth). At 16K tokens attention is about 40% of prefill FLOPs, so 5× attention sparsity bounds the speedup near 1.5×. Closest ML-side statement of the estimator-cost problem, and a direct statement of the G3 question.
Follow-ups	Minference reports 5–20% of time on index building, rising to about 30% at 10K context, on one A100. Accelerator line (IEEE HPCA/ISCA/MICRO, venues to verify): SpAtten, Sanger, ELSA, Energon, FACT; recent work reports predictor overheads above 63% of power. FlexPrefill (ICLR 2025), Twilight (NeurIPS 2025), XAttention (ICML 2025).
Added for v4 (referee citation)	DuoAttention (Xiao et al., ICLR 2025): retrieval heads keep full attention, streaming heads keep sinks and recent tokens; heads identified by a lightweight optimisation on synthetic data with frozen weights; prefill up to 1.73× (MHA) and 1.63× (GQA). Head-level, not block-level; usable here as a per-head mask.

Added for v5 (second reviewer's leads, verified)	SparKV (arXiv 2604.21231): overhead-aware KV loading on edge devices with a learned block-sparse attention latency predictor, because irregular sparsity defeats analytical models. [Withdrawn by its authors, as reported by the researcher on 4 Oct 2026; no longer cited.] CompactAttention (arXiv 2605.16839, May 2026): under chunked prefill, XAttention and SeerAttention are often slower than dense because pattern search reruns per chunk. [Read in full after v7: two causes, not one; kernels also lose efficiency when the query chunk is much shorter than the keys. XAttention end to end is 0.68× to 0.73× of dense on H200 and 0.78× to 0.84× on RTX PRO 6000, Llama-3.1-8B, TP = 2, 8K to 128K; host CPUs not reported.] AB-Sparse (arXiv 2605.12110): per-head block sizes from recall sensitivity. [Read in full after v7: decode-side, block sizes 16 to 64 at a fixed 4,096-token budget; sensitivity per head, stable across inputs; accuracy at matched recall not tested.] Oracle-guided sparse prefill (Wang et al., arXiv 2606.07703, June 2026): attention-mass top-k oracle within 1 point of dense on Qwen-family retrieval tasks. [Read in full after v7: technical report, evaluation data and code not released; the 1-point result is for per-query support at top-k 2048; support shared by 64 queries collapses at small budgets; the authors state the oracle is not a global optimum; distilled-indexer TTFT on one unnamed 80 GB GPU rises from 1.07× at 32K to 1.93× at 256K, host not reported.] Token Sparse Attention (ICML 2026, arXiv 2602.03216): prefill latency follows attention speedups on A100 and A6000.
Held (not verified)	Task-KV as IEEE TPAMI 2026; FlashPrefill V2 (arXiv 2608.19758); SparDA (arXiv 2606.04511); EntropyInfer (arXiv 2606.09508); PSA (arXiv 2503.00392); cross-layer shared routing (arXiv 2606.06467); the 0.97× vs 0.66× Token Sparse Attention figures. Not cited until read.
Added for v5 (panel)	AWS host CPUs from provider announcements: G6e (L40S) and p5.4xlarge (single H100, AMD EPYC 7R13, added August 2025) on third-generation AMD EPYC; G5 (A10G) on second-generation AMD EPYC.
Added for v6 (third referee)	nn-Meter (MobiSys 2021): kernel-level latency prediction on diverse edge devices. HELP (NeurIPS 2021): meta-learned latency prediction for unseen devices from about 10 samples. CloserToMe (Tang et al., AAAI 2026, read in full): a graph network plus a hardware capability vector and a few probe latencies on the target device predicts NAS-benchmark CNN latency on unseen GPUs, FPGAs and mobile chips; each device is one unit, with no host-device split. Transferable latency prediction for LLM screening on edge devices (arXiv 2607.21602). LLMServingSim 2.0 (Cho, Choi, Heo and Park, KAIST; IEEE ISPASS 2026, DOI 10.1109/ISPASS69572.2026.00012, read in full): serving-level simulator with per-model, per-device operator profiles and execution DAGs on ASTRA-sim. It compares against a different APEX (Lin et al., arXiv 2411.17651, a serving simulator), not the IPDPS 2026 APEX. CUDA launch queue of roughly a thousand pending launches, after which launches block (NVIDIA developer forum; to confirm in documentation or by microbenchmark).
Added for v6 (supplied by the researcher)	CTFusion: Gennai, Ohsita and Shimonishi, "A Holistic Model for End-to-End Latency Prediction of Deep Learning Tasks on Heterogeneous Systems", IEEE FLTA 2025, DOI 10.1109/FLTA67013.2025.11336244. Read in full. Closest prior work to G1. A web search had failed to find it; the earlier "not found" entry was wrong and is withdrawn.
Added: sparse attention recall of attention mass correlation with downstream accuracy proxy metric evaluation	VSPrefill (Chen, Tongji University; arXiv 2603.04460 v1; read in full): Fig. 2 shows accuracy and perplexity against attention recall on HotPotQA (functional above about 50%, indistinguishable from dense above about 90%), without stating which method produced the recall levels; Table 3 compares recall, not accuracy, across random, sampling and VSPrefill selection. Speed measured on one GPU (H20). Cross-head unified sparse attention, "Less Is More" (Yang et al.; arXiv 2508.07101 v2, under review at ICML 2026; read in full): corrects the earlier search summary, which said accuracy correlates with top-k recall. The paper treats recall as a necessary condition for long reasoning decodes, shows recall decaying over generation, and in an ablation on one AIME problem finds configurations with higher recall that still fail; it compares recall across methods, not accuracy at matched recall, and studies decoding, not prefill. The closest threat to G3; neither tests estimators at matched recall.

Group 3: methodology and measurement integrity (K6)

Query	What came back
pre-registration empirical systems evaluation	Preregistering NLP research (NAACL 2021); HCI and information-systems preregistration. No computer-systems or GPU benchmarking instance.
reproducibility GPU benchmarking methodology	HPC reproducibility survey (arXiv 2402.07530); GPU variability at scale (arXiv 2208.11035); reporting checklists. No pre-registration.
statistical significance performance measurement	Effect-size confidence intervals for performance changes (arXiv 2007.10899); Benchmarking Crimes (arXiv 1801.02381); Lilja's textbook.
measurement bias computer systems	Mytkowicz et al. (ASPLOS 2009): setup changes bias results; none of 133 surveyed papers handled it.
Follow-up	A targeted search for preregistered or registered-report computer-architecture or systems-performance studies returned none; registered reports appear in software engineering (an OSF template, arXiv 2602.09292).